

Formation Chef de Projet IA

Fondamentaux du Machine Learning

Thomas Walter Chloé-Agathe Azencott Florian Massip

Center for Computational Biology (CBIO)
Mines Paris – Institut Curie – INSERM U900
PSL Research University & PR[AI]RIE, Paris, France

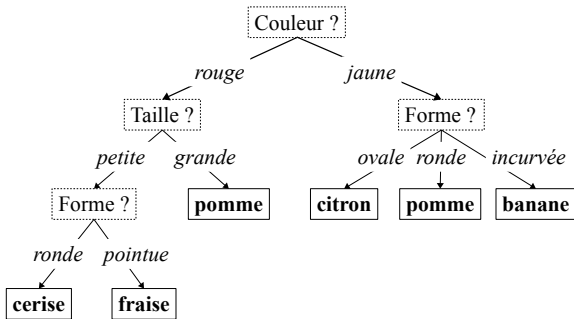
November 2022

Programme

- Introduction à l'apprentissage automatique
- **Cours 1** : Minimisation du risque empirique
- **Cours 2** : Régularisation et sélection de modèle
- **Cours 3** : Arbres et méthodes ensemblistes
- **Cours 4** : Méthodes à noyaux
- **Cours 5** : Réduction de dimension
- **Cours 6** : Clustering
- Travaux pratiques : mise en œuvre des notions et algorithmes avec scikit-learn (Python numérique).

Arbres de décision

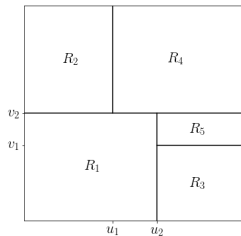
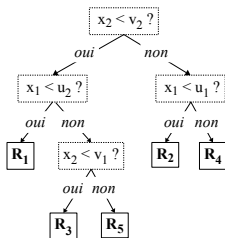
Exemple



Les arbres de décision

- Méthode hiérarchique
- Série successive de tests conditionnels.
- Souvent utilisé en dehors de la classification automatique :
 - aide à la décision
 - formalisation d'un raisonnement

Les arbres de décision pour la classification



- En classification, les tests correspondent à des seuils, appliqués aux descripteurs.
- Chaque arbre de décision correspond à une partition de l'espace des descripteurs.
- On peut assigner une classe à chaque feuille.

Les arbres de décision - avantages

- Classification en classes multiples
- Interprétabilité
- Traitement de variables catégoriques
- Mélange de types de variables

Construction des arbres

- CART: Classification and regression trees
- A chaque étape, une région est divisée en deux.
- Pour ce, il faut choisir un descripteur x_j et un seuil s .
- Ce choix est fait pour optimiser un critère, calculé sur les régions qui résultent de ce seuillage.

Split : critère de régression

- On suppose qu'un split (j, s) génère deux régions R_1 et R_2 .
- Dans le cas d'une regression, on calcule alors les moyennes $\overline{y_1}$ et $\overline{y_2}$ des étiquettes dans les régions $R_1(j, s)$ et $R_2(j, s)$.
- Le critère à minimiser par (j, s) :

$$(j, s)^* = \arg \min_{j, s} \sum_{\vec{x}_k \in R_1} (y_k - \overline{y_1})^2 + \sum_{\vec{x}_k \in R_2} (y_k - \overline{y_2})^2$$

où $\overline{y_1}, \overline{y_2}, R_1, R_2$ dépendent de (j, s) .

Split : critère de classification

- Erreur de classification :

$$Imp(R) = 1 - \max_{c=1,\dots,C} p_c(R)$$

avec $p_c(R)$ la probabilité (proportion) de la classe c dans R .

- Il s'agit de la proportion d'exemples qui n'apparait pas à la classe majoritaire.
- Ce critère est:
 - 0 si toutes les instances appartiennent à la même classe.
 - $1 - \frac{1}{C}$ si les classes sont distribuées équitablement.

Split : critère de classification

- Entropie :

$$Imp(R) = - \sum_{c=1}^C p_c(R) \log_2 p_c(R)$$

- Ce critère est:
 - 0 si toutes les instances appartiennent à la même classe.
 - $\log_2 C$ si les classes sont distribuées équitablement.

Split : critère de classification

- Critère de impureté GINI:

$$Imp(R) = \sum_{c=1}^C p_c(R)(1 - p_c(R))$$

avec $p_c(R)$ la probabilité (proportion) de la classe c dans R .

- Ce critère est:
 - 0 si toutes les instances appartiennent à la même classe.
 - $1 - \frac{1}{C}$ si les classes sont distribuées équitablement.

Split : critère de classification

- Le critère à minimiser par (j, s) :

$$(j, s)^* = \arg \min_{j, s} \frac{|R_1|}{n} Imp(R_1) + \frac{|R_2|}{n} Imp(R_2)$$

avec $|\cdot|$ le cardinal et R_1, R_2 des fonctions de (j, s) .

- On itère sur toutes les valeurs possibles de j et toutes les valeurs possibles de s pour déterminer le couple (j, s) qui minimise le critère.

Critère d'impureté GINI: Exemple

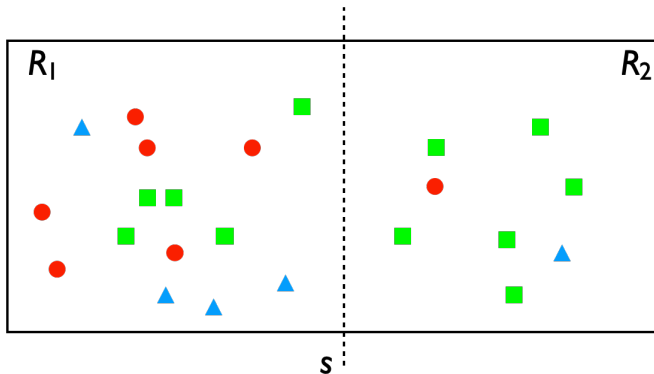


Figure: Exemple pour le calcul de l'impureté GINI

Critère d'impureté GINI: Exemple

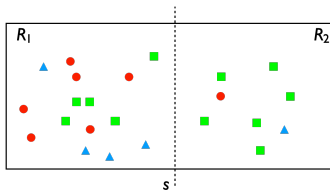


Figure: Exemple pour le calcul de l'impureté GINI

$$GI(R_1) = \frac{2}{5} \cdot \frac{3}{5} + \frac{4}{15} \cdot \frac{11}{15} + \frac{1}{3} \cdot \frac{2}{3} = 0.658$$

$$GI(R_2) = \frac{1}{8} \cdot \frac{7}{8} + \frac{1}{8} \cdot \frac{7}{8} + \frac{6}{8} \cdot \frac{2}{8} = 0.40625$$

$$GI(j, s) = \frac{15}{23}GI(R_1) + \frac{8}{23}GI(R_2) = 0.57$$

Critère d'arrêt

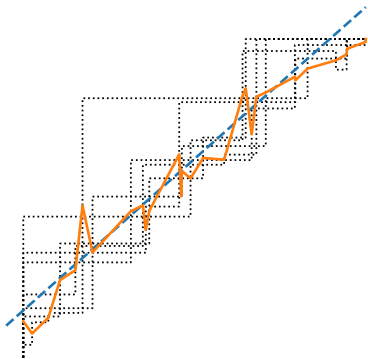
- Une classe par feuille : risque de sur-apprentissage
- Profondeur fixe
- Nombre d'observation par feuille

Avantages / Inconvénients

- Avantages :
 - Simplicité
 - Interpretabilité
 - Peut gérer des variables catégoriques et continues.
- Inconvénients :
 - Grand risque de sur-apprentissage
 - Généralement une performance limitée

2. Méthodes ensemblistes

Sagesse des foules



- Combiner des **apprenants faibles**.
- Exemple: fonction diagonale, approchée par des fonctions en esaclier.
- Principe démocratique.

Bagging

- Bagging = Bootstrap + aggregation
- On considère une partie de $\mathcal{D}$ (e.g. 80%) par échantillonnage "bootstrap" (avec remise).
- On entraîne un arbre de décision.
- On reitère cette procédure pour obtenir un ensemble d'arbres.
- Le résultat est obtenu par vote majoritaire (classification) ou en moyennant (régression).
- Faire une moyenne de modèles est effectivement une technique de régularisation.

Forêts aléatoires

- En plus du bagging, on peut à chaque noeud tirer aléatoirement des variables parmi lesquels la variable optimale est trouvée.
- Le résultat : **les forêts aléatoires**.
- Les arbres sont plus différents et couvrent "différent aspects" des données.
- En pratique :
 - généralement bons résultats sans "tuning"
 - applicable pour $P \gg N$
 - Régularisation par "ensembling"

2.4 Boosting

- Les idées :
 - tendre vers plus de complémentarité entre les classifieurs simples en se concentrant sur les erreurs
 - rajouter un terme de confiance pour chaque classifieur
- Deux variantes principales :
 - **AdaBoost** (Schapire & Freund 1997)
 - **Gradient Boosting** (Friedman 2001) : forme générale

Adaboost

- Le nom Adaboost vient de "Adaptive Boosting"
- Construction itérative : les classifieurs faibles sont forcés à se concentrer sur les erreurs du modèle
- Nous supposons pour la suite que nous avons un jeu d'entraînement $\mathcal{D} = \{(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_n, y_n)\}$ et que l'ensemble des étiquettes est $\mathcal{Y} = \{-1, 1\}$
- L'algorithme Adaboost va associer à chaque élément i de $\mathcal{D}$ un poids w_i . On a donc un jeu de données pondérées $\mathcal{D}_m = \{(w_1^m, \vec{x}_1, y_1), (w_2^m, \vec{x}_2, y_2), \dots, (w_n^m, \vec{x}_n, y_n)\}$

Adaboost

- 1 Initialisation: $w_i^1 = \frac{1}{n}$
- 2 Apprendre le classifieur f_m à partir de $\mathcal{D}_m$. Les poids influencent le calcul du critère, en remplaçant $p_c(R)$ par :

$$\tilde{p}_c(R) = \frac{1}{\sum_i w_i} \sum_{i; y_i=c} w_i \quad (1)$$

- 3 Calculer l'erreur du modèle :

$$\epsilon_m = \sum_{i=1}^n w_i^m \mathbf{1}_{f_m(\vec{x}_i) \neq y_i}$$

- 4 Calculer une confiance qu'on associe au classifieur :

$$\alpha_m = \frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m}$$

Plus l'erreur est petite, plus ce coefficient de confiance est grand.

Adaboost

- 5 On redéfinit les poids pour donner plus de poids aux échantillons sur lesquelles f_m s'est trompé :

$$w_i^{m+1} = \frac{1}{Z_m} w_i^m e^{-\alpha_m y_i f_m(\vec{x}_i)}$$

Ici, Z_m est un facteur qui fera que $\sum_i w_i^{m+1} = 1$.

- 6 Le modèle AdaBoost est alors :

$$f(\vec{x}) = \sum_{m=1}^M \alpha_m f_m(\vec{x})$$

On a donc une pondération des modèles faibles (des arbres) avec ses coefficients de confiance.

Gradient Boosting

- AdaBoost est en fait un cas particulier d'une famille de techniques appelees *boosting du gradient* (*gradient boosting en anglais*).
- On considere la fonction de perte exponentielle :

$$\begin{aligned} \{-1, 1\} \times \mathbb{R} &\rightarrow \mathbb{R} \\ y, f(\vec{x}) &\mapsto e^{-yf(\vec{x})} \end{aligned}$$

- et la fonction de décision cummulative :

$$F_M(\vec{x}) = \sum_{m=1}^M \alpha_m f_m(\vec{x})$$

- On peut montrer alors que la règle de AdaBoost minimise cette fonction de coût.

Conclusion

- Les forêts agrègent des résultats de classifieurs faibles
- Stratégies :
 - Forêt aléatoire repose sur la diversification et le vote
 - AdaBoost focalise sur les erreurs (approche collaborative)
- AdaBoost est souvent sensible au bruit d'étiquette, mais meilleur pour les jeux de données "propres"
- Les Forêts aléatoires sont souvent plus robustes.
- Ajouter des arbres est habituellement avantageux pour les forêts aléatoires (jusqu'à 500), mais pas nécessairement pour AdaBoost.